

1. Información General

- Proyecto:** Laboratorio Técnico U2 - Configuración de Pipeline CI/CD.
- Autores:** Carlos Alfonso Muñoz Agudelo, Esteban Giovanny Garay Cano, Carlos Sebastian Castillo Silva.
- Fecha:** Junio de 2026.
- Institución:** Universidad de la Sabana - Maestría en Arquitectura de Software.

2. Introducción y Objetivos

El proyecto implementa una metodología de aprendizaje basado en proyectos (ABP) para automatizar la construcción, prueba y validación de software bajo estándares industriales.

- Objetivo CI:** Automatizar procesos de compilación y ejecución de **20 pruebas unitarias** para asegurar retroalimentación inmediata ante cambios en el repositorio.
- Objetivo CD:** Definir un pipeline robusto ("Pipeline-as-Code") con Jenkins para la construcción de imágenes de contenedores y su publicación en registros como DockerHub.

3. Stack Tecnológico

La selección tecnológica se basa en criterios de interoperabilidad y escalabilidad:

- Lenguaje y Framework:** ASP.NET Core 10 y Entity Framework Core 10.
- Base de Datos:** PostgreSQL (alojada en Supabase).
- Integración Continua (CI):** GitHub Actions (integración nativa y baja latencia).
- Entrega Continua (CD):** Jenkins (flexibilidad y orquestación agnóstica).
- Contenedores:** Docker (portabilidad) y Kubernetes/GKE (orquestación final).

4. Desarrollo Técnico

4.1. Aplicación y API

El microservicio gestiona clientes (EcomifyCustomers) mediante una arquitectura CRUD.

- Endpoints:** Incluye métodos GET, POST, PUT y DELETE para la gestión de la tabla **ecommify_customers**.

- Pruebas Unitarias:** Se implementaron 20 pruebas con **xUnit y Moq**, validando lógica de modelos y servicios (ej. validación de direcciones y prevención de duplicados).

4.2. Containerización (Docker)

Se utiliza un **Dockerfile multi-etapa** para optimizar el tamaño de la imagen final:

- 1.**Build Stage:** Utiliza el SDK 10.0 de .NET para restaurar dependencias y publicar el binario.

- 2.**Runtime Stage:** Utiliza la imagen **aspnet:10.0** (más ligera) para ejecutar la aplicación en el puerto 8080.

- 3.**Comando de construcción:** **docker build -t ecomify-customers:latest -f EcomifyCustomers/Dockerfile EcomifyCustomers**.

4.3. Pipeline CI (GitHub Actions)

Configurado en **.github/workflows/ci.yml**.

- Triggers:** Se activa ante **push** o **pull_request** en la rama **main**, o manualmente via **workflow_dispatch**.

- Etapas (Jobs):**

- xUnit Tests:** Realiza el checkout, instala .NET 10, restaura paquetes, compila y ejecuta las 20 pruebas, generando reportes en formato TRX y cobertura OpenCover.

- SonarQube Analysis:** Realiza un análisis estático de código para detectar deuda técnica y errores de seguridad (ejecución condicionada al éxito de las pruebas).

4.4. Pipeline CD (Jenkins)

Utiliza un archivo **Jenkinsfile** para definir el flujo de despliegue.

- Proceso:** Clona el repositorio, construye la imagen Docker y la publica en **DockerHub (estebangaraycano/ecomify-customers)** y **Google Artifact Registry**.

- Justificación:** Permite un manejo profesional de despliegues complejos en clústeres de Kubernetes (GKE).

5. Conclusiones

- Sinergia:** La combinación de GitHub Actions para CI (agilidad) y Jenkins para CD (robustez) optimiza el ciclo de vida del software.

- Calidad (Shift-Left):** La automatización de pruebas y el análisis con SonarQube permiten detectar vulnerabilidades de forma temprana.

- Portabilidad:** El uso de Docker y Kubernetes garantiza la consistencia de la aplicación en cualquier entorno de nube